

W 2

Python 程式設計基礎

虛擬環境 · 讀懂第一支爬蟲 · 例外處理

吳彥霖 博士

Python 爬蟲實務應用 · 第 2 週 · 09/22

三節課，三件事

節 1

虛擬環境

- 概念、建立、啟用
- 管理：更新 / 刪除 / 升級重建

節 2

Python 程式基礎

- requests / read_html / 寫檔

節 3

例外處理

- try / except 為什麼需要

節 1

虛擬環境

Virtual Environment

虛擬環境

定義

在專案資料夾裡，建立一個自我獨立的 Python 環境；裝進去的套件只屬於這個環境，不影響全域、也不被別的專案影響。

參考文件：

- [venv — Creation of virtual environments](#)

主要概念

- 為每個專案，建立各自專屬的 Python 開發/執行環境。
- 它裝什麼套件，都只留在專案裡

生活類比

- 每個專案配一個專屬工具箱
- 工具（套件）各自放置，不跟別的專案混用

venv 官方文件摘要

- 專案隔離：

為特定專案提供專屬的 Python 直譯器與套件庫，避免不同專案間的套件衝突。

- 慣用目錄：

通常建立在專案下的 `.venv` 或 `venv` 資料夾，或集中管理於 `~/.virtualenvs` 等容器目錄。

- 免版本控制：

不要把虛擬環境目錄加進 Git 等原始碼版本控制系統。

- 具可拋棄性：

視為可隨時刪除、重建的消耗品；請勿把任何專案原始碼放在環境目錄內。

- 不可移動與複製：

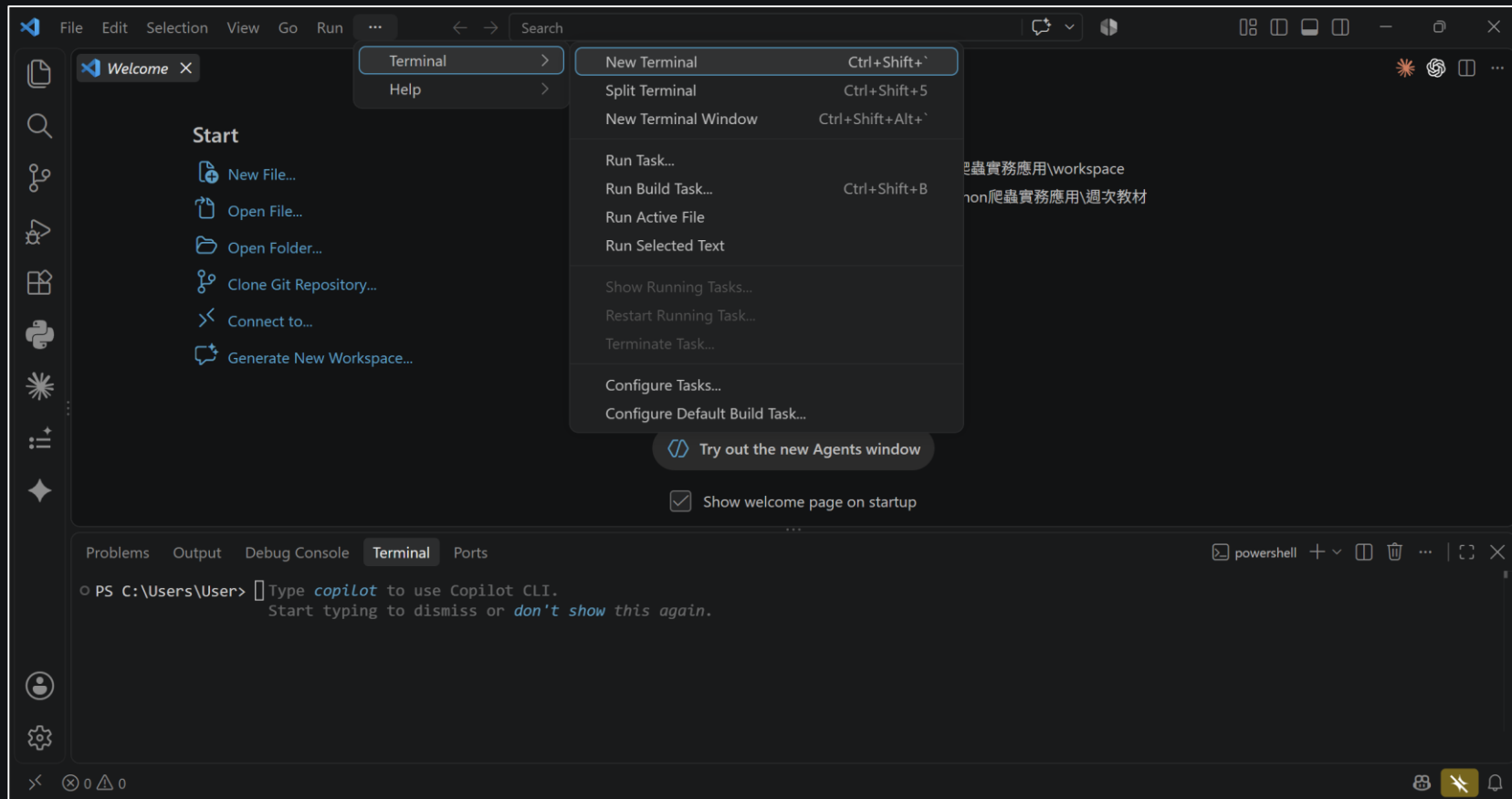
建立後無法直接搬到其他路徑；若要更換位置，必須在目標位置重新建立。

參考文件：

■ [venv — Creation of virtual environments](#)

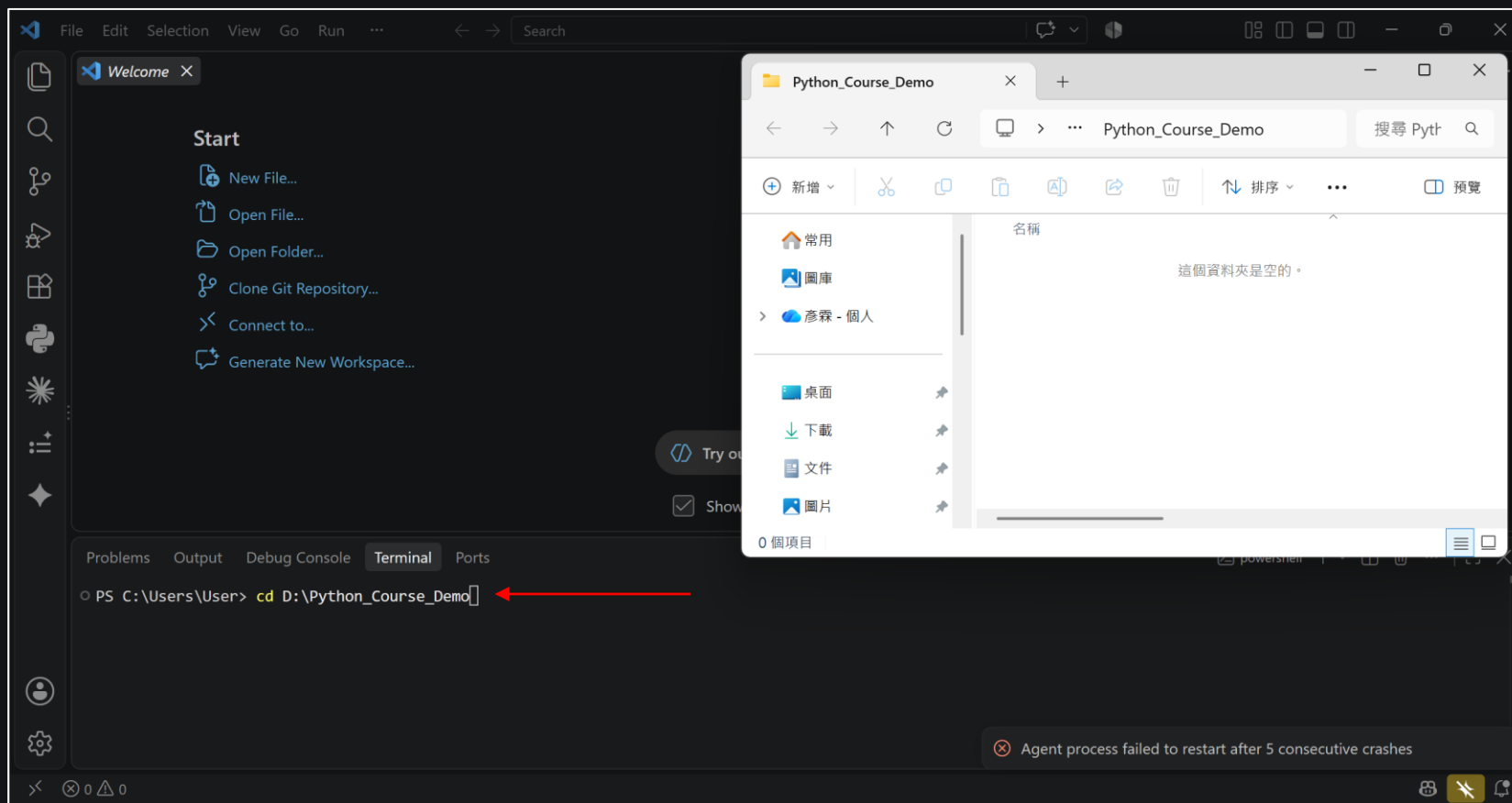
建立虛擬環境

Step 1 : 開啟 New Terminal



建立虛擬環境

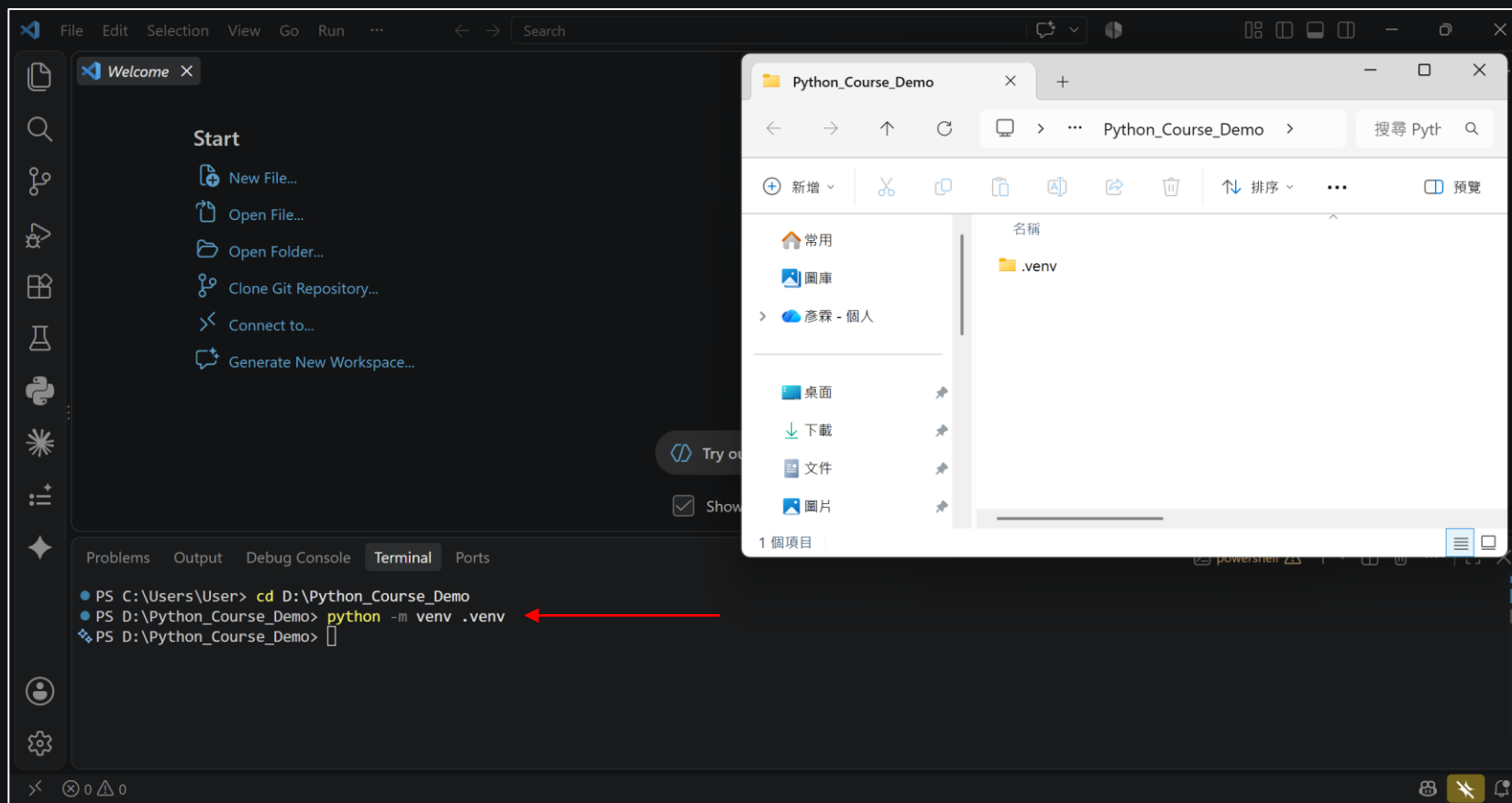
Step 2 :切換到專案資料夾目錄中



建立虛擬環境

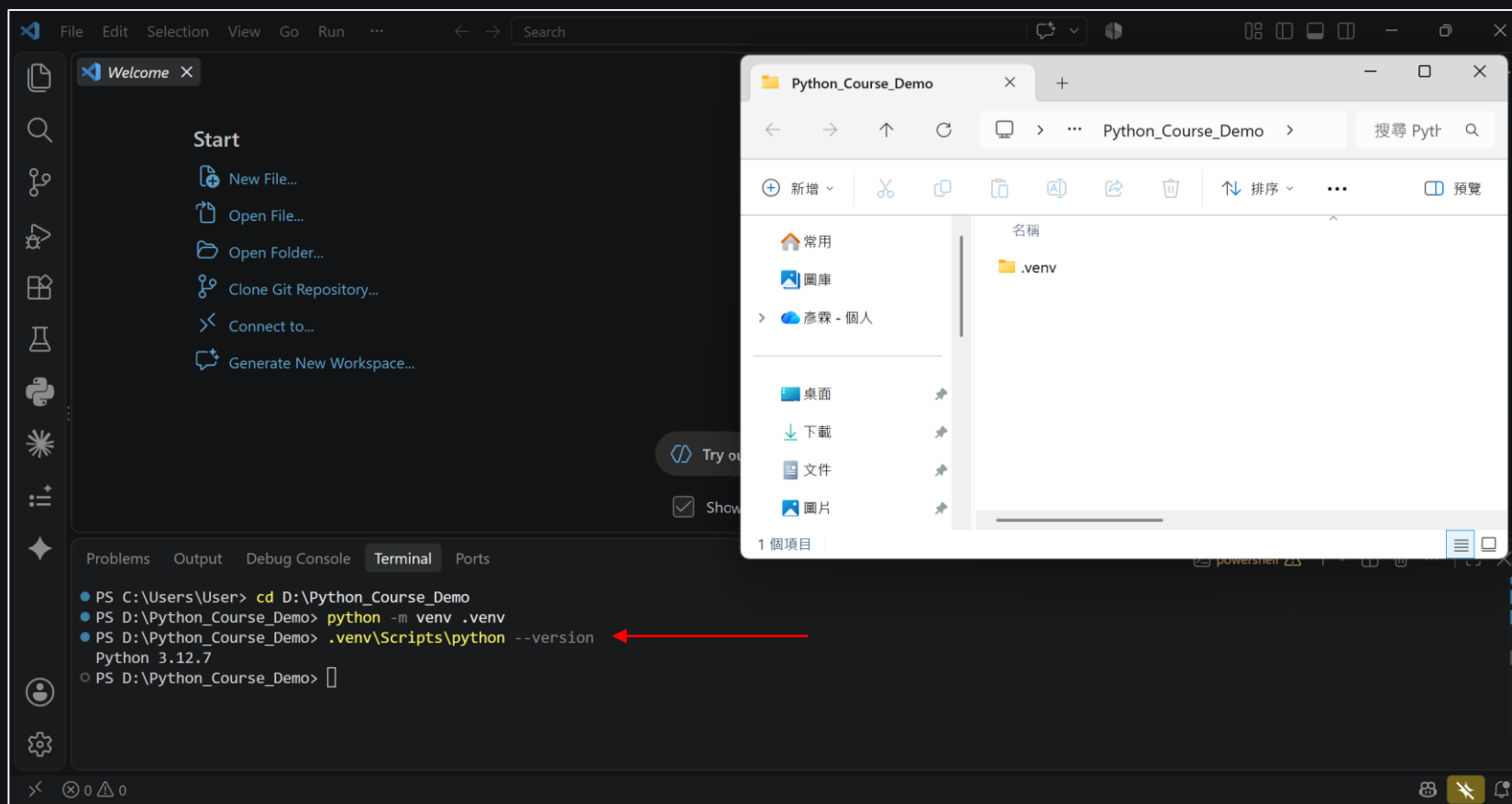
Step 3 : 專案資料夾中創建 .venv 子資料夾 `python -m venv .venv`

相對路徑，表示虛擬環境的儲存位置



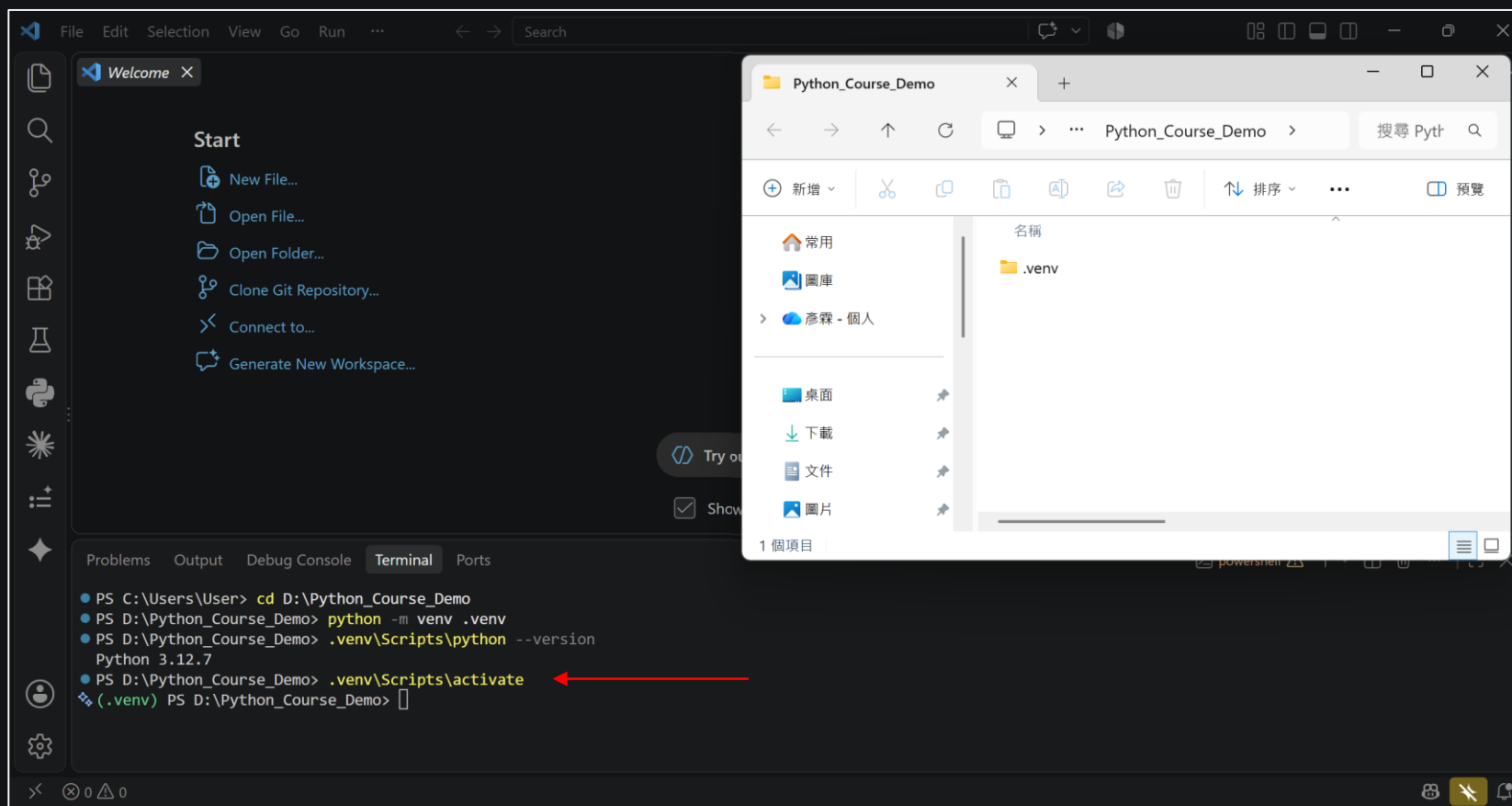
建立虛擬環境

Step 4 : 查詢 Python 版本



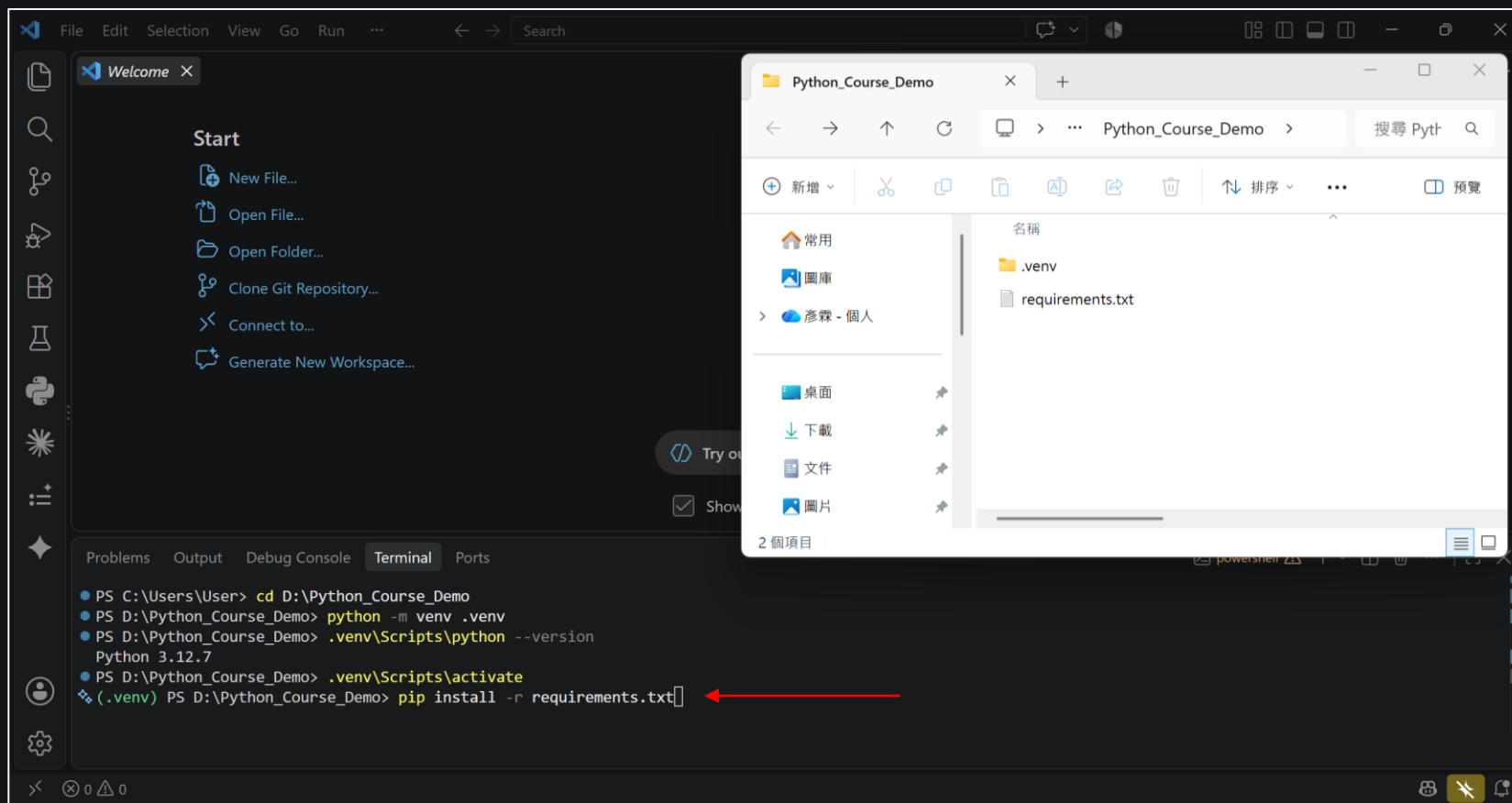
建立與啟動虛擬環境

Step 5 : 啟動虛擬環境 `.venv\Scripts\activate`



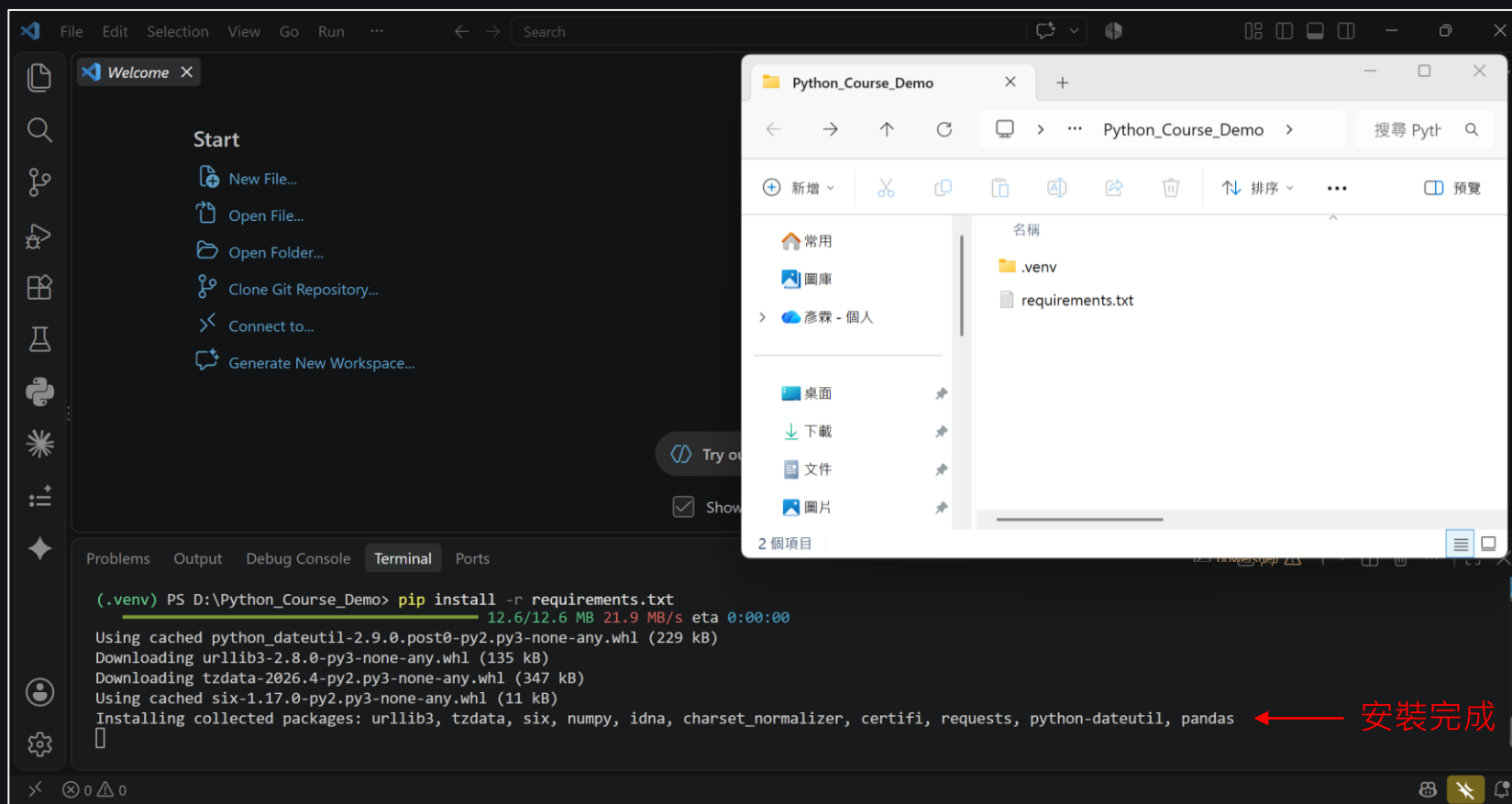
虛擬環境中安裝 Python 套件

Step 6 : 安裝 Python 套件 `pip install -r requirements.txt`



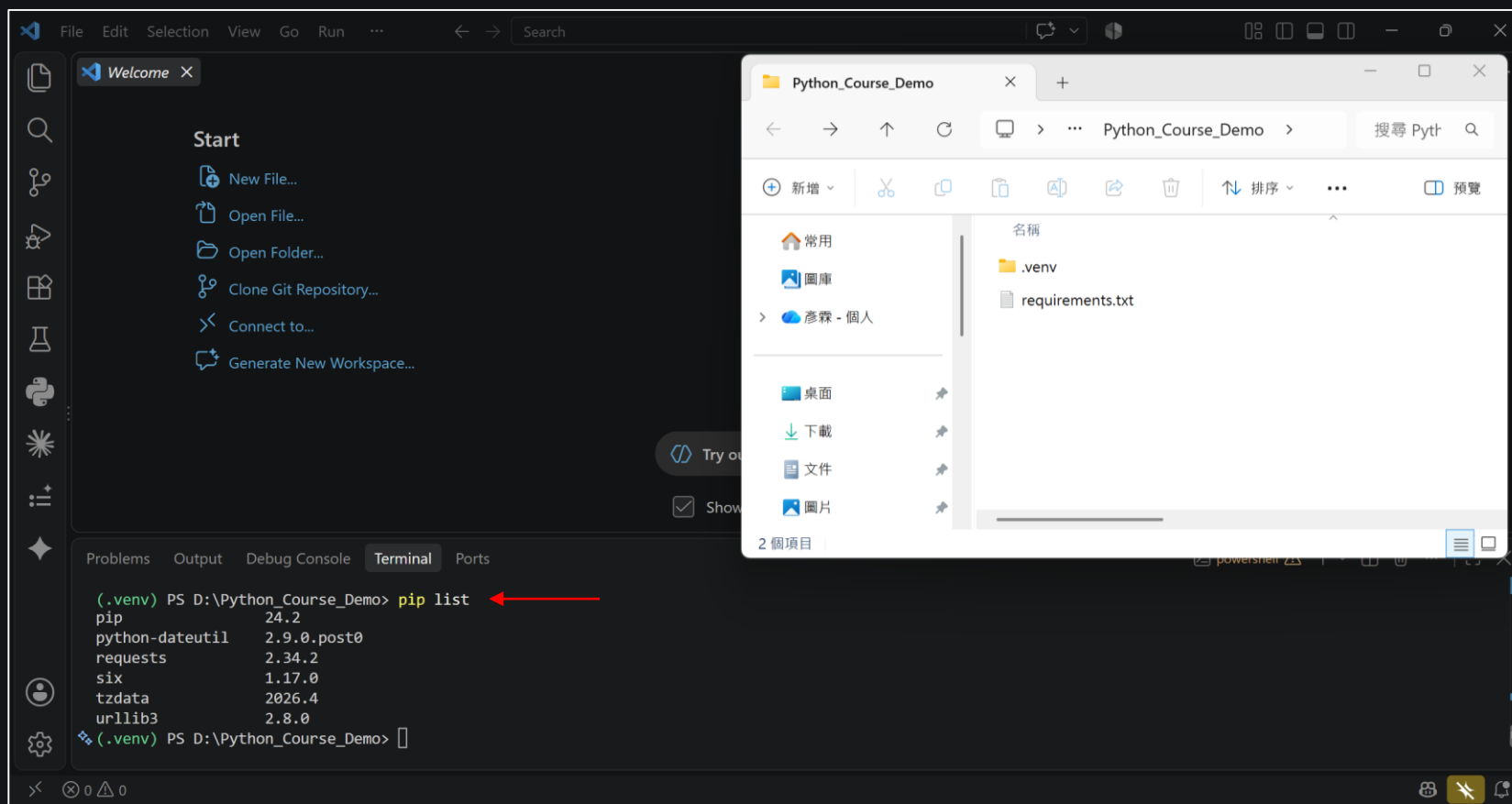
虛擬環境中安裝 Python 套件

Step 6 : 安裝 Python 套件



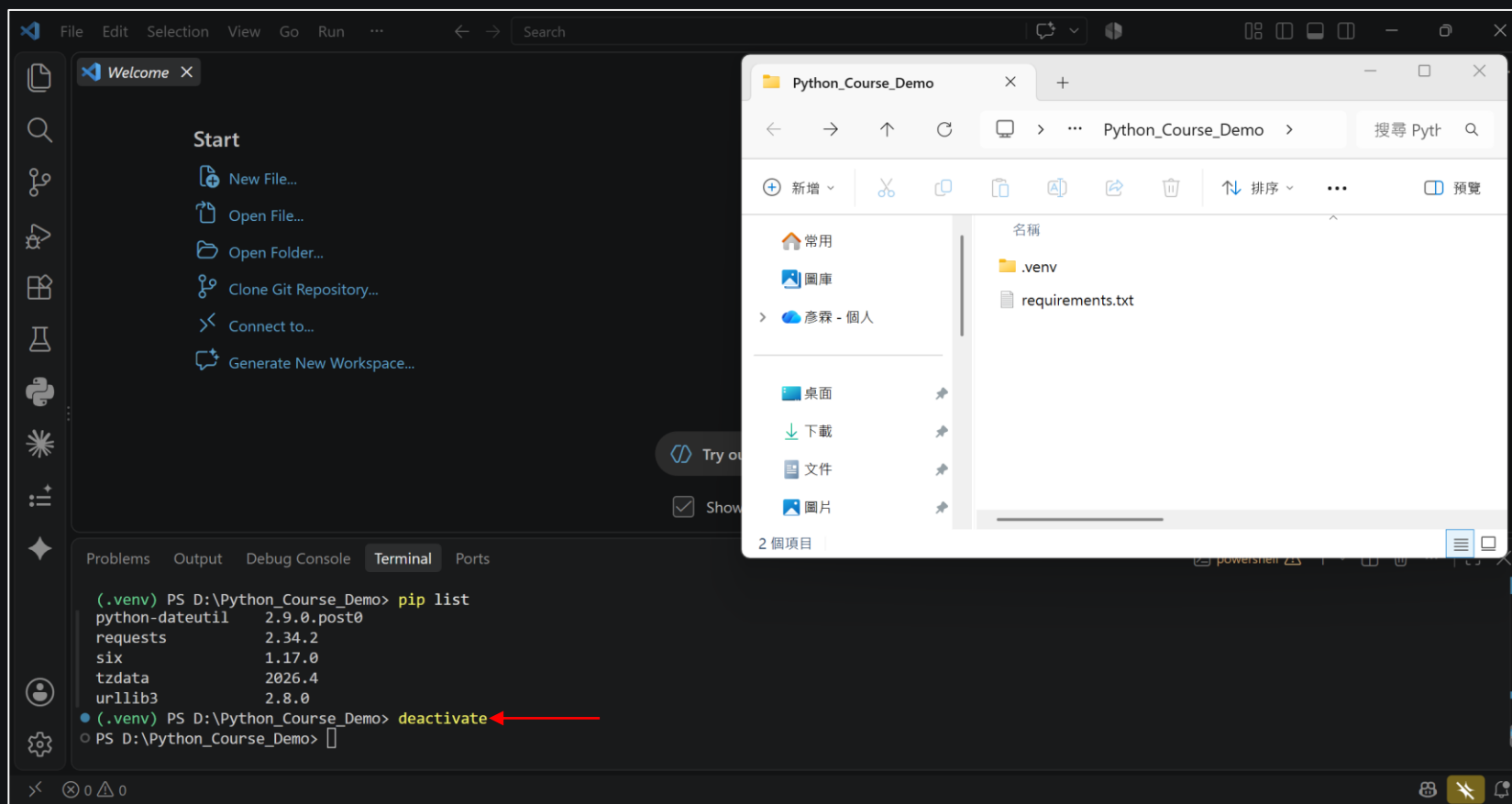
建立與啟動虛擬環境

Step 7 : 檢視已安裝的 Python 套件與版本 **pip list**



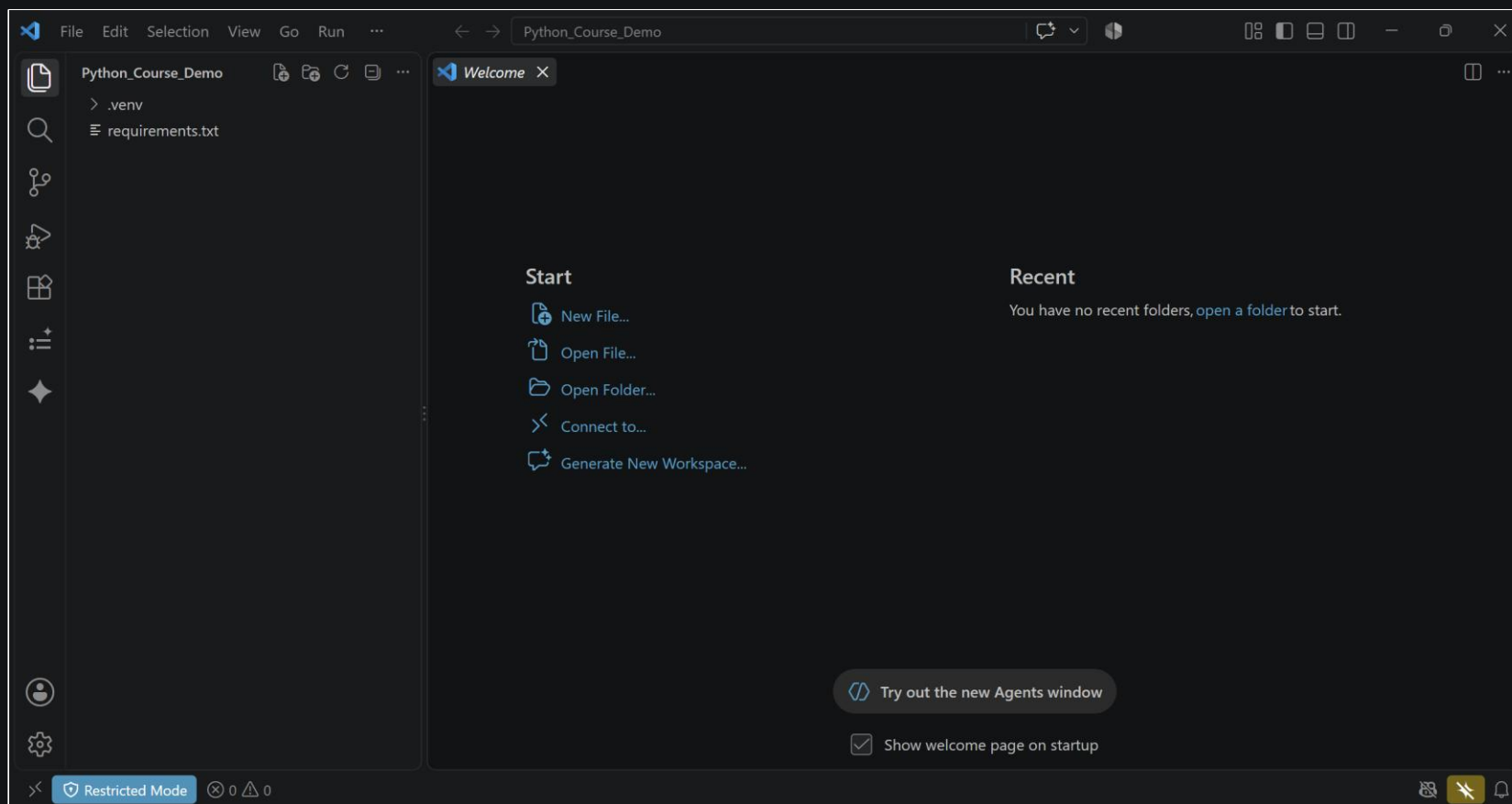
離開虛擬環境

Step 8 : 離開虛擬環境 **deactivate**



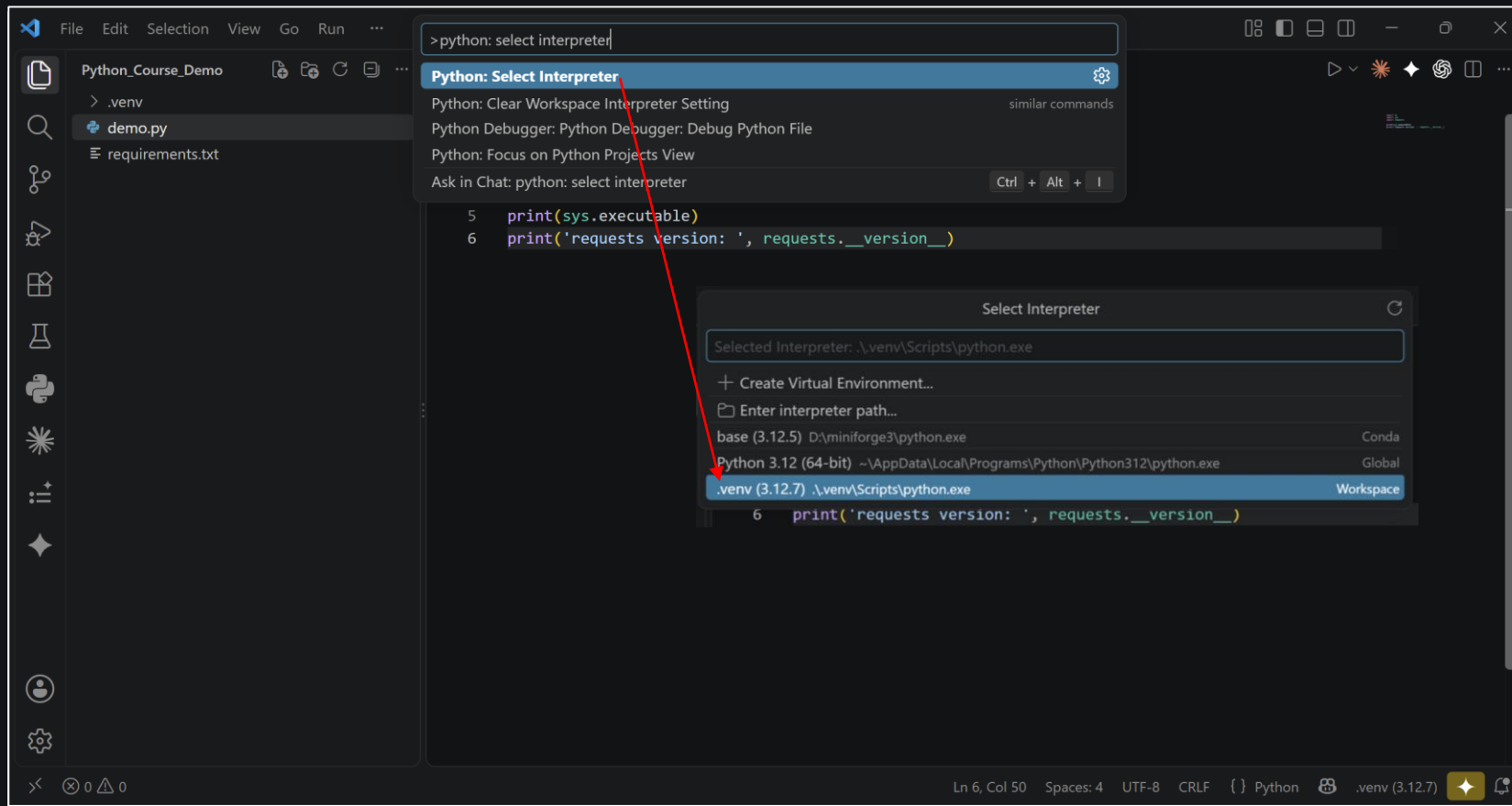
開啟專案資料夾

Step 9 : VS Code 點擊 Explorer 選擇專案資料夾



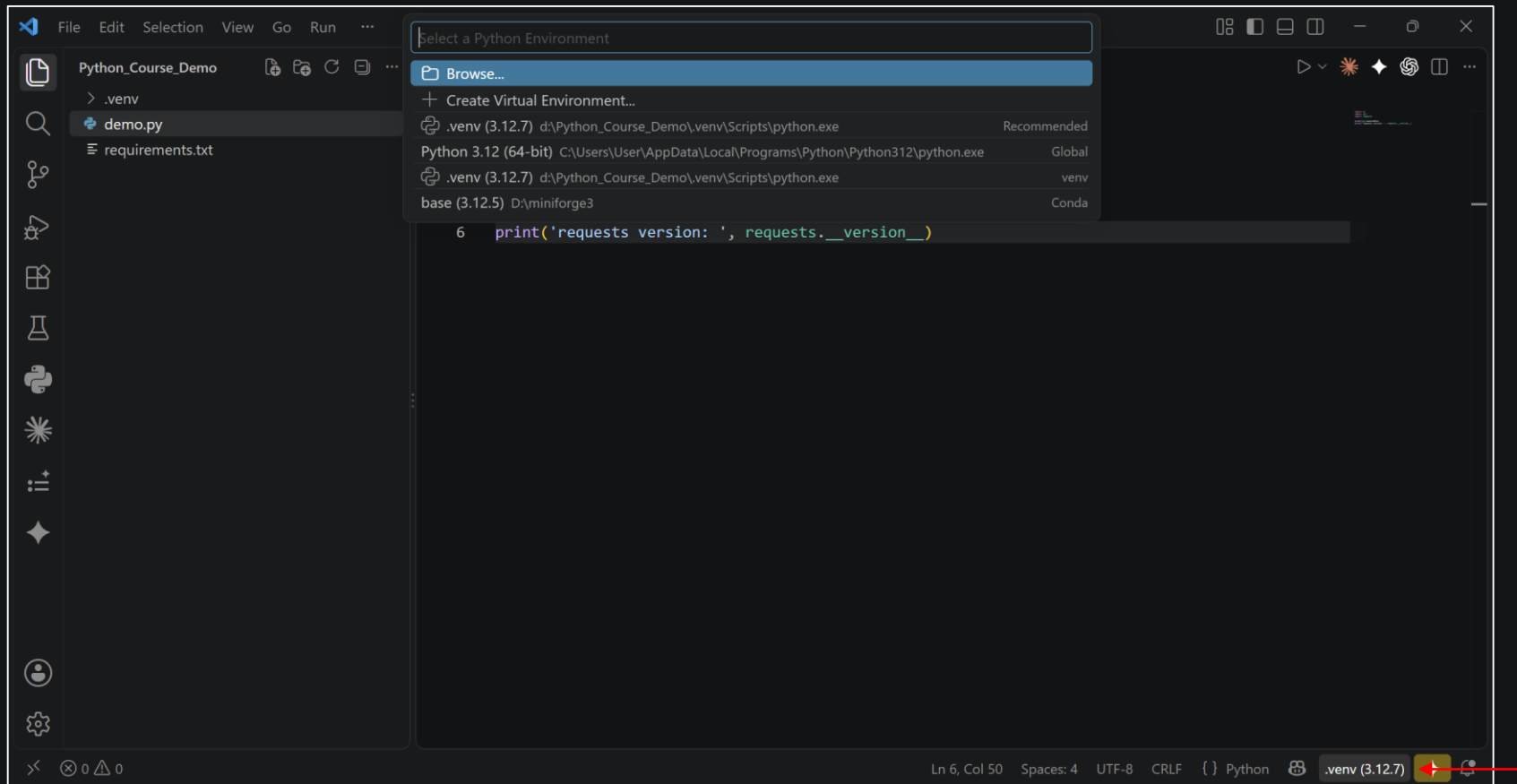
選擇 Python 直譯器

Step 10 : Ctrl+Shift+P → Python: Select Interpreter → 選擇專案直譯器



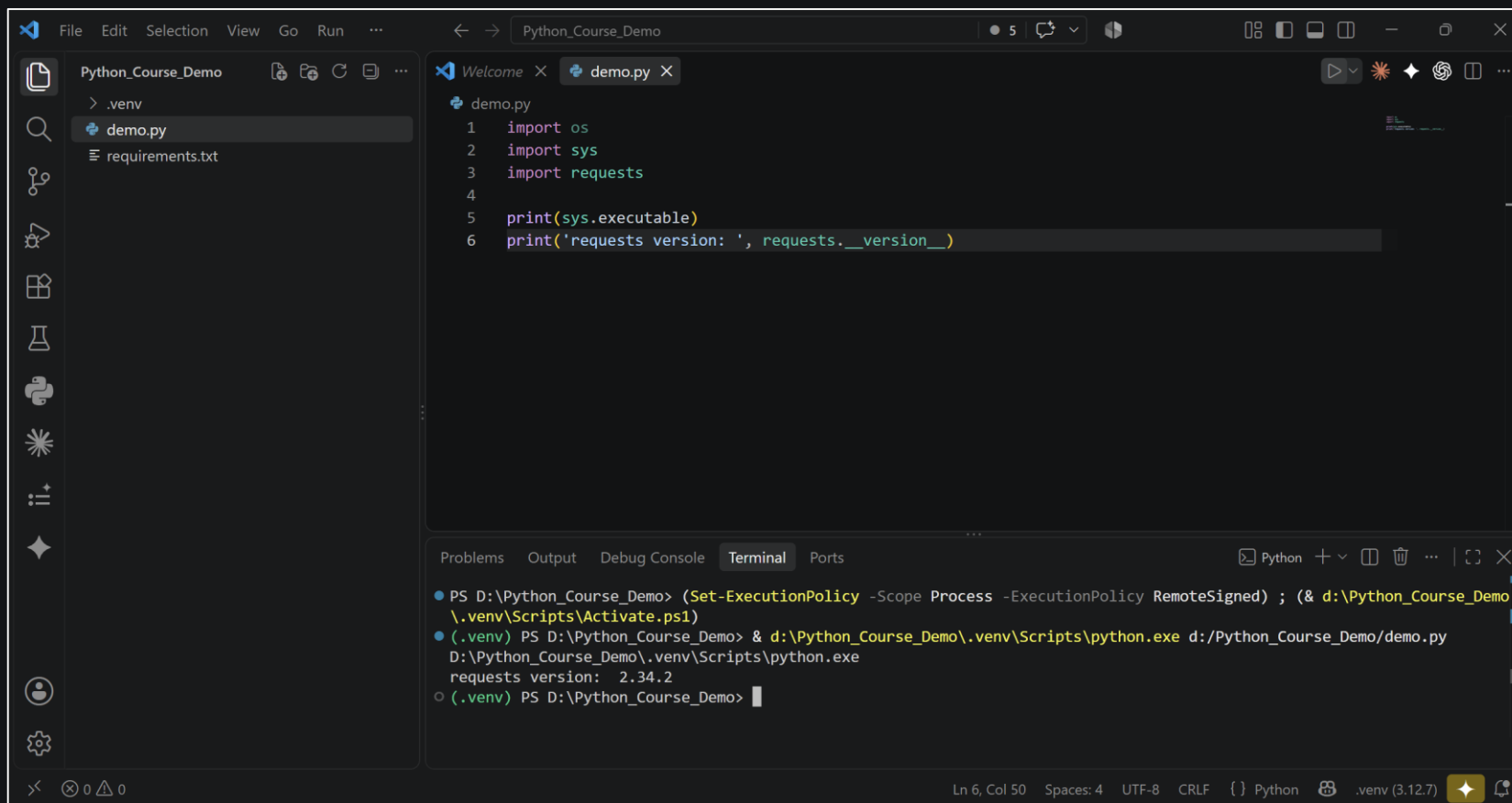
切換 Python 直譯器

Step 11: 開啟 .py 檔，選擇此專案 Python 直譯器



撰寫執行 Python 程式碼

Step 12: 執行 .py 檔



The screenshot shows the Visual Studio Code interface with a project named 'Python_Course_Demo'. The file explorer on the left shows a '.venv' directory containing 'demo.py' and 'requirements.txt'. The editor window displays the 'demo.py' file with the following code:

```
1 import os
2 import sys
3 import requests
4
5 print(sys.executable)
6 print('requests version: ', requests.__version__)
```

The terminal at the bottom shows the execution of the script. The commands and output are as follows:

```
PS D:\Python_Course_Demo> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\Python_Course_Demo\venv\Scripts\Activate.ps1)
(.venv) PS D:\Python_Course_Demo> & d:\Python_Course_Demo\venv\Scripts\python.exe d:\Python_Course_Demo/demo.py
D:\Python_Course_Demo\venv\Scripts\python.exe
requests version: 2.34.2
(.venv) PS D:\Python_Course_Demo>
```

The status bar at the bottom indicates the file is at line 6, column 50, using UTF-8 encoding with CRLF line endings. The Python version is 3.12.7, and the virtual environment is active.

課堂練習 - 1

承接前人的專案：重建一模一樣的環境

情境

你接手一位前人開發的專案。它需要 **Python 3.14**，且用到以下套件版本。請把環境重建到「能執行」。

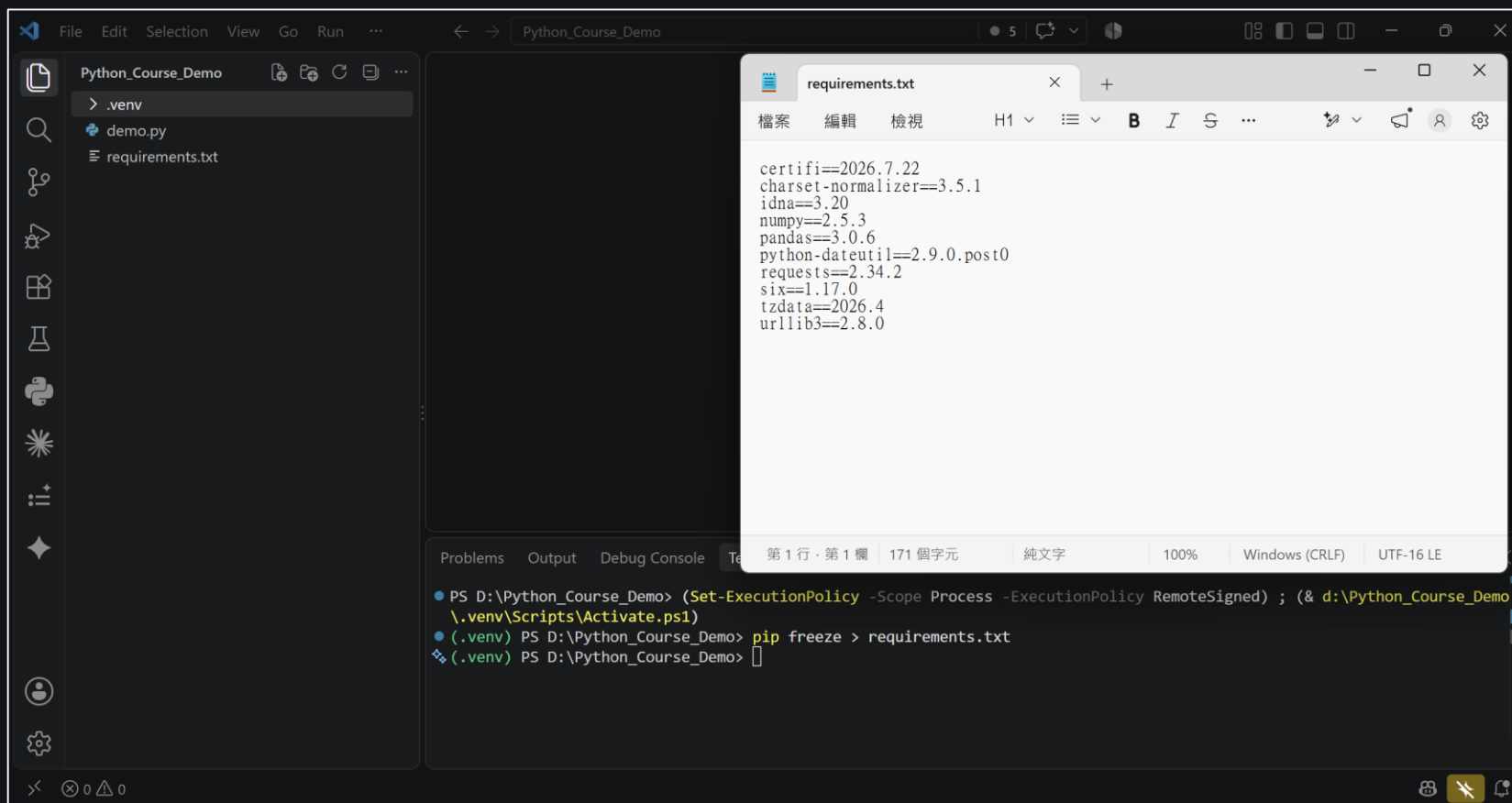
```
numpy==2.4.4  
pandas==2.3.3  
scipy==1.17.1  
plotly==6.7.0  
yfinance==1.2.2  
gradio==6.12.0
```

步驟

- 1 先到 python.org 裝 Python 3.14
- 2 開專案資料夾 → Terminal ▶ New Terminal
- 3 `py -3.14 -m venv .venv` （用 3.14 建立）
- 4 `.venv\Scripts\activate` → `python --version` 要顯示 3.14
- 5 `pip install -r requirements.txt`
- 6 驗收：`pip list` 對版本、`import` 套件不報錯

Python 套件版控

將套件版本寫進 requirements.txt 中 `pip freeze > requirements.txt`



The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows the file structure of the 'Python_Course_Demo' project, including a '.venv' folder, 'demo.py', and 'requirements.txt'. The main editor area displays the 'requirements.txt' file, which contains the following text:

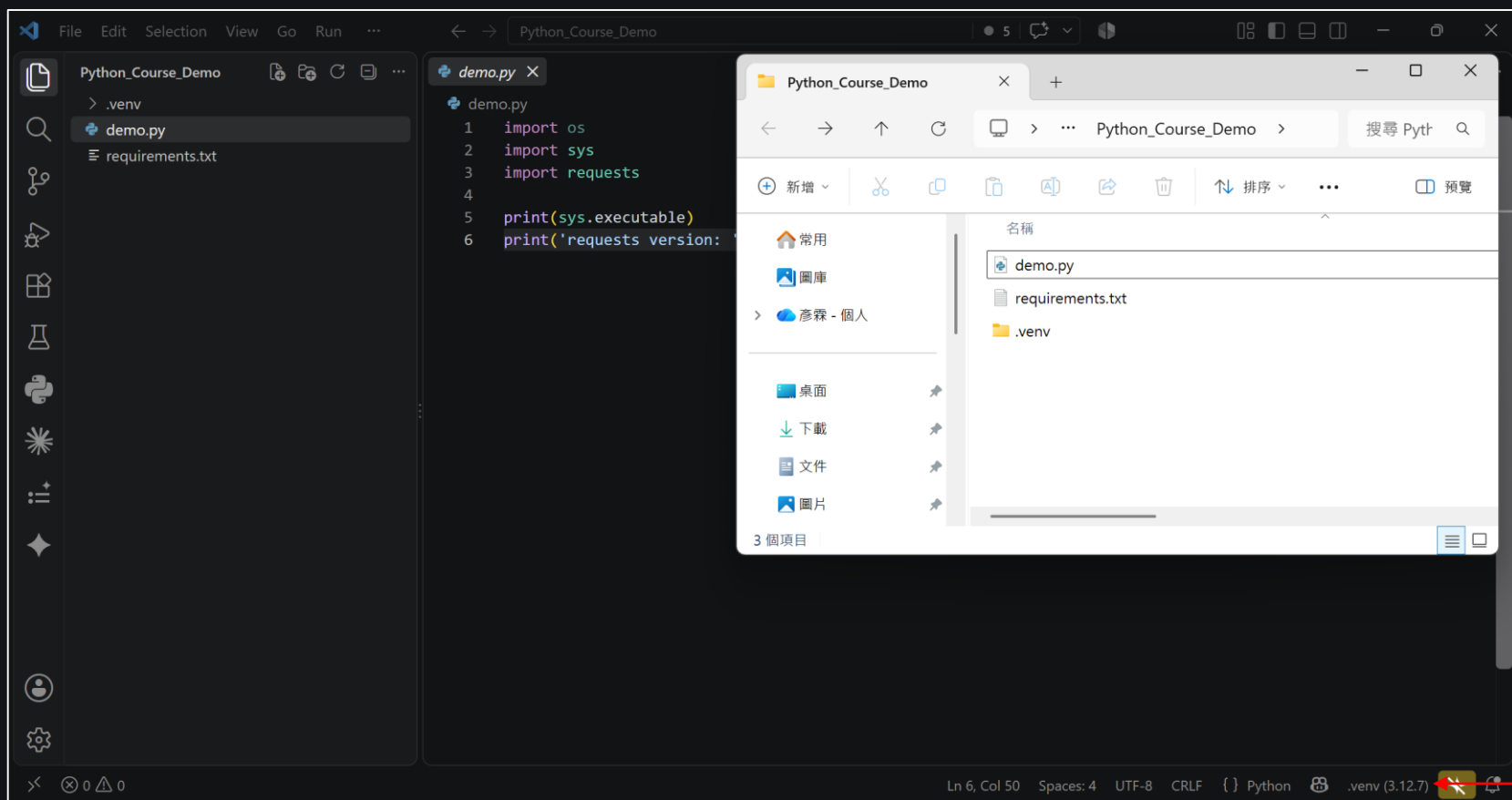
```
certifi==2026.7.22
charset-normalizer==3.5.1
idna==3.20
numpy==2.5.3
pandas==3.0.6
python-dateutil==2.9.0.post0
requests==2.34.2
six==1.17.0
tzdata==2026.4
urllib3==2.8.0
```

Below the editor, the Output pane shows the terminal output of the command `pip freeze > requirements.txt` executed in the `.venv` environment:

```
PS D:\Python_Course_Demo> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& d:\Python_Course_Demo\venv\Scripts\Activate.ps1)
(.venv) PS D:\Python_Course_Demo> pip freeze > requirements.txt
(.venv) PS D:\Python_Course_Demo>
```

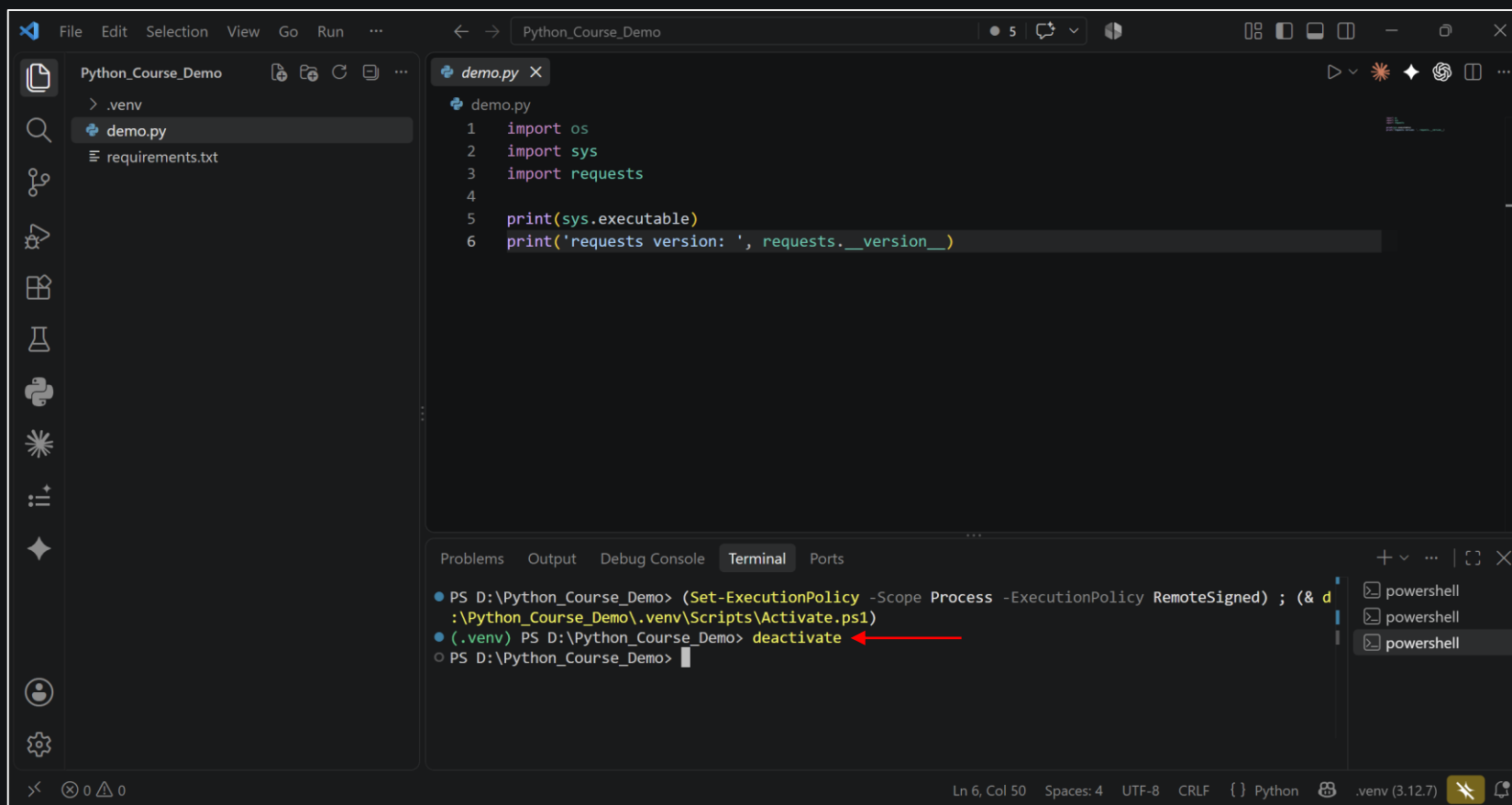
刪除虛擬環境

Step 0 : 當前虛擬環境 .venv(3.12.7)



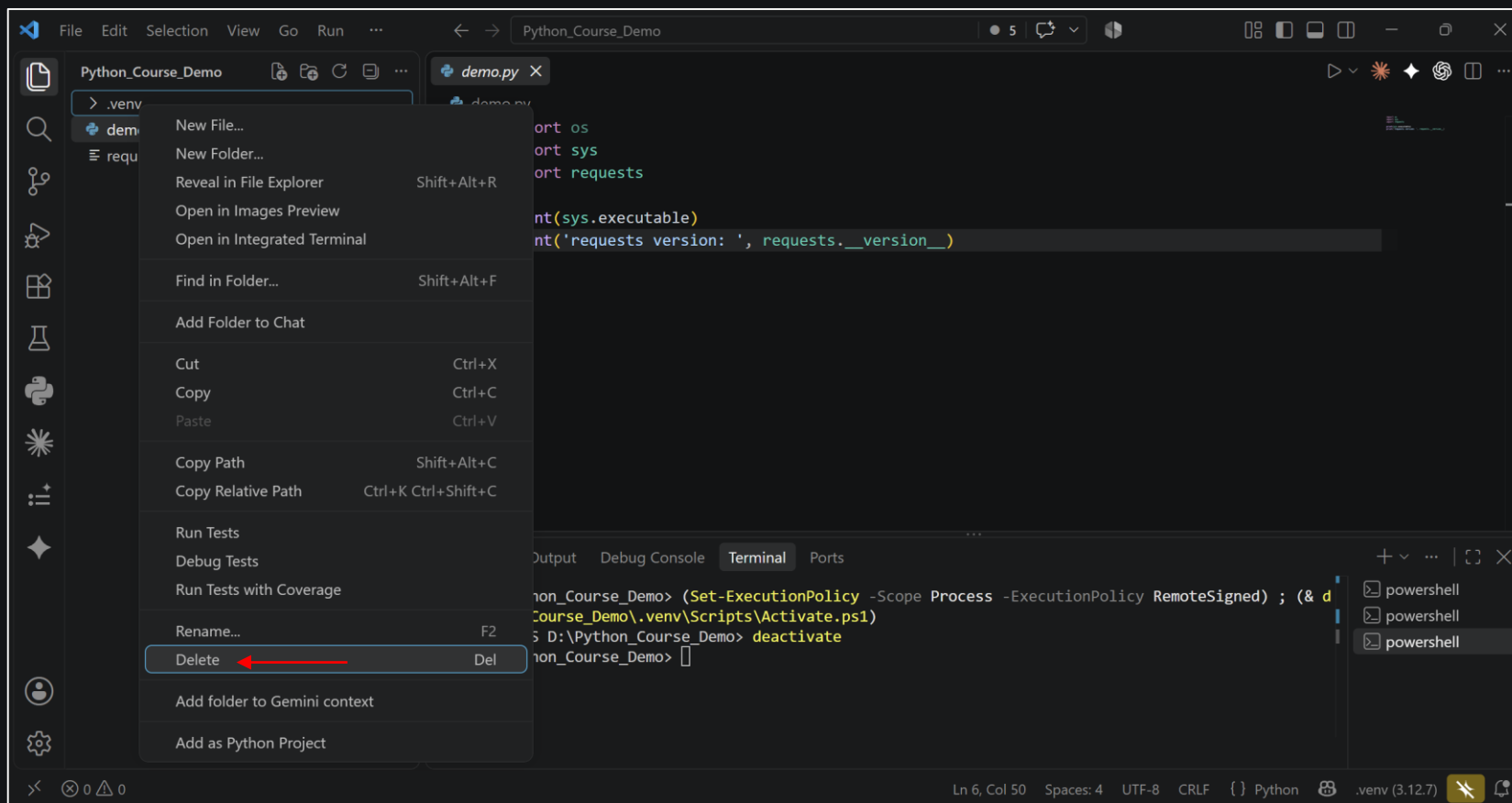
刪除虛擬環境

Step 1 : 離開虛擬環境 **deactivate**



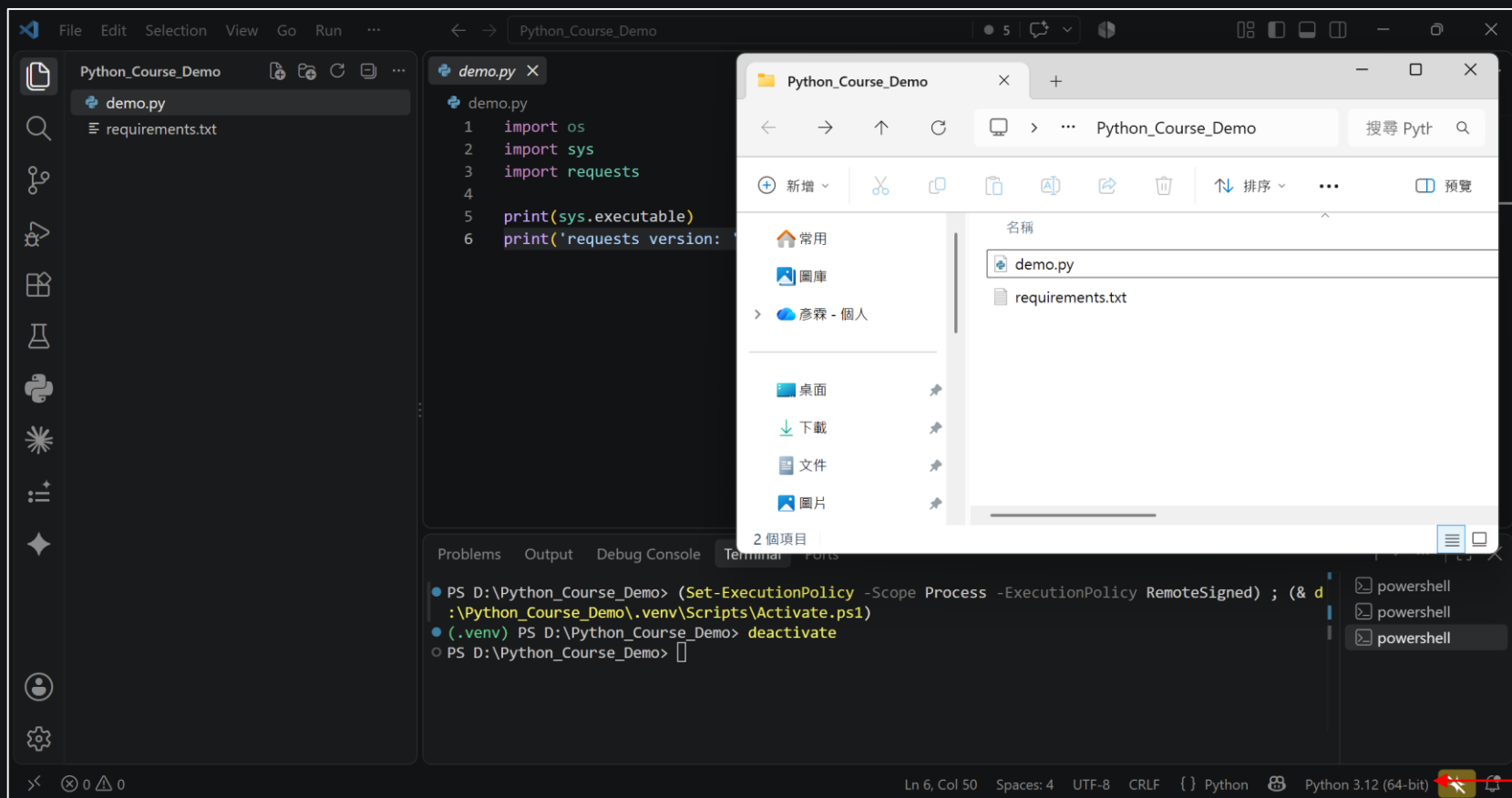
刪除虛擬環境

Step 2 : 點擊 .venv 資料夾 → 點擊 Delete



刪除虛擬環境

Step 3: 虛擬環境 .venv 已刪除



Python 升級了新版本，專案環境想跟著更新，

該怎么做？

節 2

Python 程式基礎

Client-Server 架構



requests 是一個幫我們「送出請求、收下回應」的工具

瀏覽器其實也是在做同一件事，只是它把回應畫成漂亮的網頁，然而 requests 把回應的原始內容(HTML 文字)交給程式來處理。

摘要 爬蟲 = 用程式模擬「送請求、收回應」，再從回應裡取出要的資料。

Python 程式架構

Python 爬蟲的程序

① 給網址

URL / HEADERS

② 取內容

`requests.get` → .text

③ 解析

`pd.read_html`

④ 存檔

`to_csv` / `to_excel`

三個關鍵工具

`requests` (取得資料) 、 `pandas.read_html` (解析資料) 、 寫檔 (儲存資料)

Python 程式架構

```
import requests
import pandas as pd
from io import StringIO
import os

URL = "https://example.com/some_table_page"
HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CourseBot/1.0"
}

resp = requests.get(URL, headers=HEADERS, timeout=20)
resp.raise_for_status()
print(f"{URL} ( 狀態碼 {resp.status_code} ) ")
html = resp.text

tables = pd.read_html(StringIO(html))
df = tables[0]

file_name = os.path.join(os.path.dirname(__file__), "output")
df.to_csv(file_name + ".csv", index=False, encoding="utf-8-sig") # CSV
df.to_excel(base + ".xlsx", index=False) # Excel ( 需 openpyxl )
```

GET / POST 請求

```
import requests
import pandas as pd
from io import StringIO
import os

URL = "https://example.com/some_table_page"
HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CourseBot/1.0"
}

resp = requests.get(URL, headers=HEADERS, timeout=20)
resp.raise_for_status()
print(f"{URL} ( 狀態碼 {resp.status_code} ) ")
html = resp.text

tables = pd.read_html(StringIO(html))
df = tables[0]

file_name = os.path.join(os.path.dirname(__file__), "output")
df.to_csv(file_name + ".csv", index=False, encoding="utf-8-sig") # CSV
df.to_excel(base + ".xlsx", index=False) # Excel ( 需 openpyxl )
```

- .get() 取得資料
- .post() 傳送資料
- HEADERS (標頭) 就像是包裹外箱上的「寄件託運單」或信封上的「附註資訊」。
當我們透過程式與網站伺服器溝通時，除了傳送或接收實際的資料 (包裹內的物品)，還必須夾帶一些「溝通用的詮釋資料 (Metadata)」，來幫助雙方了解該如何處理這批訊息。
- HEADERS 包含以下幾種具體用途：
 - 身份表明 (User-Agent)
告訴伺服器我們是誰。如果沒有特別設定，requests 發出請求時，預設會顯示 python-requests/x.x.x。許多網站的安全機制(例如：防火牆、防爬蟲系統等)一看到這個明顯的程式特徵，就會直接拒絕連線(403)。
 - 資料格式 (Content-Type)
說明傳遞的資料是什麼格式，例如：JSON、純文字或圖片檔案。
 - 權限認證 (Authorization / Cookie)
夾帶 API 密鑰或登入狀態，向伺服器證明我們有權限讀取或修改這些資料。

STATUS_CODE 狀態碼

```
import requests
import pandas as pd
from io import StringIO
import os

URL = "https://example.com/some_table_page"
HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CourseBot/1.0"
}

resp = requests.get(URL, headers=HEADERS, timeout=20)
resp.raise_for_status()          # 狀態碼
print(f"{URL} ( 狀態碼 {resp.status_code} ) ")
html = resp.text

tables = pd.read_html(StringIO(html))
df = tables[0]

file_name = os.path.join(os.path.dirname(__file__), "output")
df.to_csv(file_name + ".csv", index=False, encoding="utf-8-sig") # CSV
df.to_excel(base + ".xlsx", index=False)                       # Excel ( 需 openpyxl )
```

- 狀態碼 5 類：

100 ~ 199 資訊回應

200 ~ 299 成功回應

300 ~ 399 重新導向

400 ~ 499 用戶端錯誤回應

500 ~ 599 伺服器錯誤回應

- 200 OK

請求成功；GET 資源已被擷取並在訊息主體中傳送。

- 404 Not Found

伺服器找不到所請求的資源。在瀏覽器中，這意味著 URL 不被識別；在 API 中，這也可能表示端點是有效的，但資料本身不存在。

參考資料：

■ [Overview of Status Codes](#)

取得物件字串解碼

```
import requests
import pandas as pd
from io import StringIO
import os

URL = "https://example.com/some_table_page"
HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CourseBot/1.0"
}

resp = requests.get(URL, headers=HEADERS, timeout=20)
resp.raise_for_status()          # 狀態碼
print(f"{URL} ( 狀態碼 {resp.status_code} ) ")
html = resp.text                # 回應內容 ( 解碼成字串 )

tables = pd.read_html(StringIO(html))
df = tables[0]

file_name = os.path.join(os.path.dirname(__file__), "output")
df.to_csv(file_name + ".csv", index=False, encoding="utf-8-sig") # CSV
df.to_excel(base + ".xlsx", index=False)                        # Excel ( 需 openpyxl )
```

- `requests.get()` 回傳一個裝了成功從伺服器拿到的所有東西 (狀態碼、標頭、內容...)，`resp.text` 是將取得到的內容主體，解碼成字串 (`str`)。
- `resp.text` 所解碼的字串內容，等同我們在瀏覽器按 `Ctrl+U` 「檢視網頁原始碼」看到的那一大段文字。
- 解碼後若出現中文亂碼，可嘗試下列方式：

```
resp.encoding = "utf-8" # 指定正確編碼
```

網頁表格轉 DataFrame 清單

```
import requests
import pandas as pd
from io import StringIO
import os

URL = "https://example.com/some_table_page"
HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CourseBot/1.0"
}

resp = requests.get(URL, headers=HEADERS, timeout=20)
resp.raise_for_status()          # 狀態碼
print(f"{URL} ( 狀態碼 {resp.status_code} ) ")
html = resp.text                 # 回應內容 ( 解碼成字串 )

tables = pd.read_html(StringIO(html))
df = tables[0]

file_name = os.path.join(os.path.dirname(__file__), "output")
df.to_csv(file_name + ".csv", index=False, encoding="utf-8-sig") # CSV
df.to_excel(base + ".xlsx", index=False)                       # Excel ( 需 openpyxl )
```

- `pd.read_html()` 將網頁原始碼中所有 `<table>` 都解析成 DataFrame，回傳一個 DataFrame 的清單(list)。
- `StringIO()` 把這段字串包成一個「類檔案物件」，換句話說，就是讓一段文字假裝成一個檔案，可以被當檔案來讀。

DataFrame 寫入 CSV / EXCEL

```
import requests
import pandas as pd
from io import StringIO
import os

URL = "https://example.com/some_table_page"
HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CourseBot/1.0"
}

resp = requests.get(URL, headers=HEADERS, timeout=20)
resp.raise_for_status()          # 狀態碼
print(f"{URL} ( 狀態碼 {resp.status_code} ) ")
html = resp.text                 # 回應內容 ( 解碼成字串 )

tables = pd.read_html(StringIO(html))
df = tables[0]

file_name = os.path.join(os.path.dirname(__file__), "output")
df.to_csv(file_name + ".csv", index=False, encoding="utf-8-sig") # CSV
df.to_excel(base + ".xlsx", index=False)                        # Excel ( 需 openpyxl )
```

- 寫入成 EXCEL (.xlsx 檔) 需再安裝 openpyxl 套件，因為，EXCEL 為特定二進位結構的格式，pandas 寫 .xlsx 的預設引擎為 openpyxl。
- `os.path.dirname(__file__)`
此 .py 檔的所在資料夾路徑
- `os.path.join(資料夾路徑, 檔名)`
用分隔符將資料夾路徑與檔名接起來

CSV 與 EXCEL 的差別

```
df.to_csv(path, index=False, encoding="utf-8-sig") # CSV  
df.to_excel(path, index=False) # Excel ( 需 openpyxl )
```

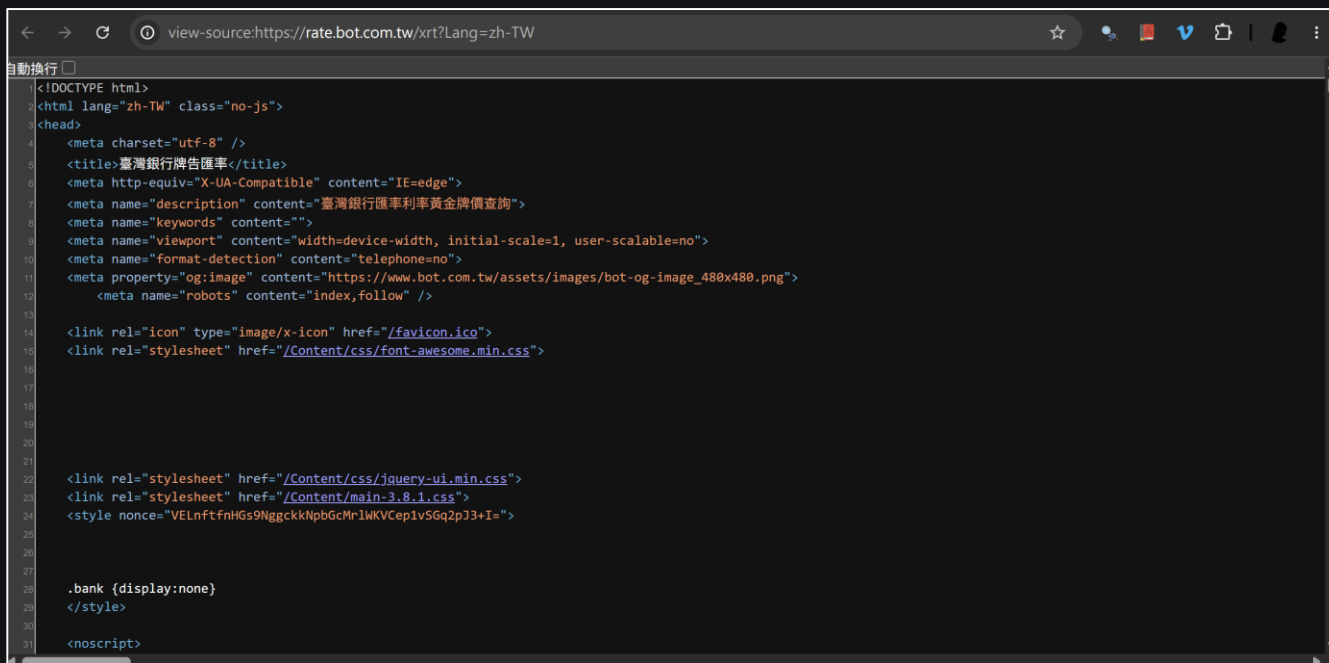
CSV (純文字)

- 萬用、任何工具都能開
- 免額外套件
- 中文用 utf-8-sig，Excel 開不亂碼
- 缺點：不能存格式/多工作表

Excel (.xlsx)

- 真正的 Excel 檔，可多工作表/格式
- 需要裝 openpyxl 套件
- 適合直接交給非技術同事看
- 缺點：檔案較大、非純文字

判斷 read_html 是否可用?



```
<!DOCTYPE html>
<html lang="zh-TW" class="no-js">
<head>
  <meta charset="utf-8" />
  <title>臺灣銀行牌告匯率</title>
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="description" content="臺灣銀行匯率利率黃金牌價查詢">
  <meta name="keywords" content="">
  <meta name="viewport" content="width=device-width, initial-scale=1, user-scalable=no">
  <meta name="format-detection" content="telephone=no">
  <meta property="og:image" content="https://www.bot.com.tw/assets/images/bot-og-image_480x480.png">
  <meta name="robots" content="index, follow" />

  <link rel="icon" type="image/x-icon" href="/favicon.ico">
  <link rel="stylesheet" href="/Content/css/font-awesome.min.css">

  <link rel="stylesheet" href="/Content/css/jquery-ui.min.css">
  <link rel="stylesheet" href="/Content/main-3.8.1.css">
  <style nonce="VELnftfnHG59NggckkNpbGCMr1wKVcep1vSgq2p3+I=">

  .bank {display:none}
</style>

<noscript>
```

三步判斷

- 1 右鍵 → 檢視網頁原始碼 (Ctrl+U)
- 2 Ctrl+F 搜尋 <table
- 3 Ctrl+F 搜尋 表格裡的數據是否存在

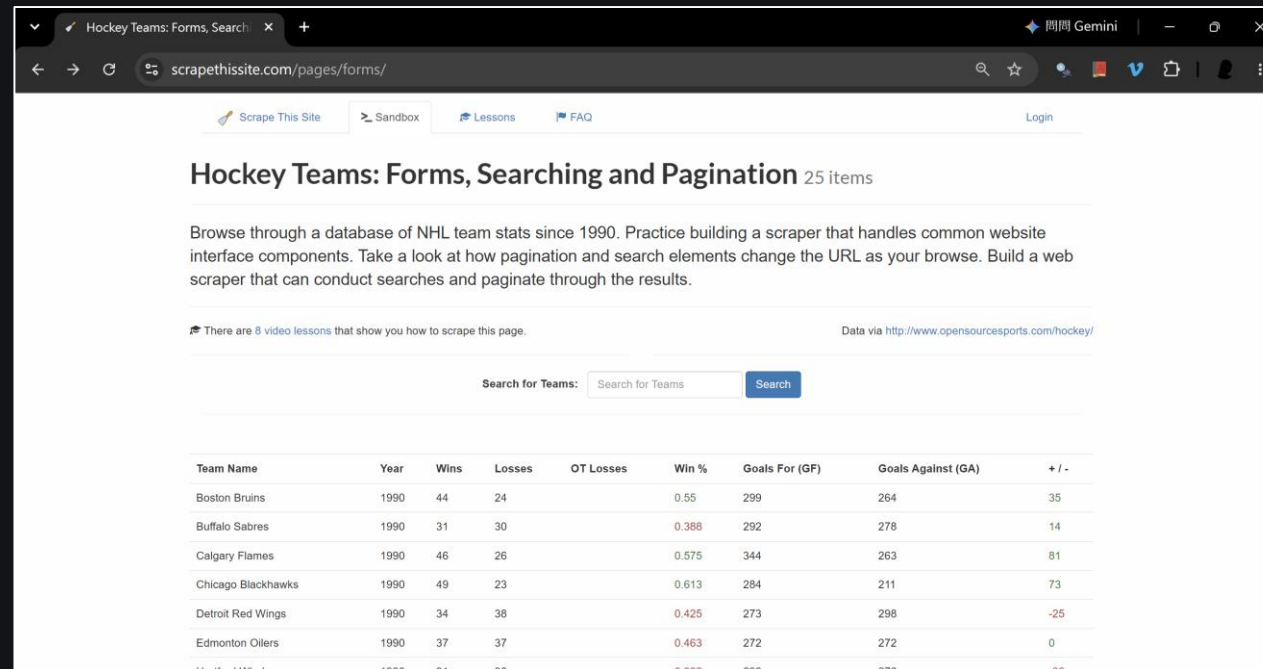
都找得到 → ✓ 用 read_html

搜不到真值 / 只有一堆 <script> → ✗ JS 動態，改走 API (W8)

訣竅 資料在「原始碼」裡找得到，read_html 才有機會。

課後練習 - 2

- 爬取 ScrapeThisSite 練習網站的冰球隊伍統計表
- 將表格資料存成 hockey_teams.xlsx 檔
- URL | <https://www.scrapethissite.com/pages/forms/>



The screenshot shows a web browser window with the URL <https://www.scrapethissite.com/pages/forms/>. The page title is "Hockey Teams: Forms, Searching and Pagination" with 25 items. Below the title is a description: "Browse through a database of NHL team stats since 1990. Practice building a scraper that handles common website interface components. Take a look at how pagination and search elements change the URL as your browse. Build a web scraper that can conduct searches and paginate through the results." There is a search bar with the text "Search for Teams:" and a "Search" button. Below the search bar is a table of NHL team statistics.

Team Name	Year	Wins	Losses	OT Losses	Win %	Goals For (GF)	Goals Against (GA)	+/-
Boston Bruins	1990	44	24		0.55	299	264	35
Buffalo Sabres	1990	31	30		0.388	292	278	14
Calgary Flames	1990	46	26		0.575	344	263	81
Chicago Blackhawks	1990	49	23		0.613	284	211	73
Detroit Red Wings	1990	34	38		0.425	273	298	-25
Edmonton Oilers	1990	37	37		0.463	272	272	0
Hartford Whalers	1990	21	38		0.288	238	276	-38

課後練習 - 3

- 爬取臺灣銀行本日營業時間牌告匯率
- 將本日牌告費率存成 20260922.xlsx 檔
- URL | <https://rate.bot.com.tw/xrt?Lang=zh-TW>



The screenshot shows the Bank of Taiwan's exchange rate page for 2026/09/22. It includes a header with the bank's logo and name, a search bar, and a language selector. Below the header, there is a section for the current date's exchange rates, followed by a table of exchange rates for various currencies. The table has columns for currency type, cash rates (buy/sell), forward rates (buy/sell), and historical rates. The currencies listed are USD, HKD, GBP, AUD, CAD, SGD, and CHF.

2026/09/22 本行營業時間牌告匯率

請注意：

1. 本表資料僅供參考，不代表實際交易匯率。
2. 「網路銀行」及「Easy儲蓄上申請現鈔」之實際交易匯率，以交易時顯示之匯率為準。
3. 離櫃實際交易匯率以交易時本行匯率為準。
4. 本網頁牌告匯率資訊為靜態顯示，顯示之牌告匯率資訊不會隨後續變動而自動更新資訊，欲得知本行最新牌告匯率資訊請按「取得最新報價」。
5. 本頁面匯率資訊於本行營業時間後顯示「本行營業時間牌告匯率」，於本行非營業時間後顯示「本行非營業時間匯率」，如欲查詢本行歷史匯率資訊，請至「歷史匯率查詢(另開視窗)」。
6. 本行提供線上申請外幣現鈔服務並享有優惠匯率，如有需求可至「Easy儲外幣現鈔網站(另開視窗)」。
7. 貴銀人壽提供線上購買旅遊平安險服務，如有需求可至「E好保旅行平安保險(另開視窗)」。

取得最新報價 匯率試算 線上申請外幣現鈔

牌價最新掛牌時間：2026/09/22 15:09

幣別	現金匯率		即期匯率		遠期匯率	歷史匯率
	本行買入	本行賣出	本行買入	本行賣出		
美金 (USD)	31.295	31.965	31.645	31.745	查詢	查詢
港幣 (HKD)	3.884	4.088	4.01	4.07	查詢	查詢
英鎊 (GBP)	41.17	43.29	42.18	42.58	查詢	查詢
澳幣 (AUD)	22.15	22.93	22.44	22.64	查詢	查詢
加拿大幣 (CAD)	22.1	23.01	22.5	22.7	查詢	查詢
新加坡幣 (SGD)	24.26	25.17	24.75	24.93	查詢	查詢
瑞士法郎 (CHF)	37.8	39	38.48	38.73	查詢	查詢

節 3

例外處理 try / except

什麼是例外處理？

try / except 能夠在發生錯誤時進行對應的動作，不僅能保護整個程式的流程，也能夠掌握問題出現的位置，馬上進行修正。

範例程式碼

```
import numpy as np

x = 5
y = 100/x

if y > 50:
    print("y > 50")
else:
    print("y <= 50")
```



```
import numpy as np

x = 0
y = 100/x

if y > 50:
    print("y > 50")
else:
    print("y <= 50")
```

ZeroDivisionError: division by zero

什麼是例外處理？

try / except 能夠在發生錯誤時進行對應的動作，不僅能保護整個程式的流程，也能夠掌握問題出現的位置，馬上進行修正。

範例程式碼

```
import numpy as np

x = 5
y = 100/x

if y > 50:
    print("y > 50")
else:
    print("y <= 50")
```



```
import numpy as np

try:
    x = 0
    y = 100 / x

    if y > 50:
        print("y > 50")
    else:
        print("y <= 50")

except ZeroDivisionError:
    print("x 不能為 0")
```

例外類型

常見例外類型

- ValueError：值不合法，如 `int("abc")`
- ZeroDivisionError：除以 0
- TypeError：型別不對，如 `"a"+1`
- KeyError：dict 沒有那個鍵
- IndexError：list 索引超出範圍
- FileNotFoundError：找不到檔案

不知道是哪一個？這樣寫

- 先跑一次，看 Traceback 最後一行的名字
- 或先「廣接」印出真正的錯誤名稱：

```
try:  
    ...  
except Exception as e:  
    print(type(e).__name__, e)
```

- 查到真名後，再改成精準的 `except`

參考資料：

- [Built-in Exceptions](#)

課堂練習 - 3

任務

下面為一份學生的成績，但是，混了缺漏的資料（空字串）。
請用 `try/except` 跳過缺漏的資料，最後只印出有效資料的個數與加總。

```
scores = ["80", "90", "", "", "60", "100"]
```

預期輸出

```
有效 4 筆，總計 = 330
```

今天，我們學到了什麼？

下週見 · Python 爬蟲實務應用